# Software Requirements Specification

# CareConnect

# Table of Contents

# 1. Introduction

CareConnect is a digital platform designed to connect people in need of social support with volunteers, donors, and organizations willing to help. It manages requests related to handicap assistance, orphanage aid, blood donation, and old-age home care in a structured and transparent manner.

## 1.1 Purpose:

The purpose of CareConnect is to provide a centralized system where users can request help, volunteers and donors can fulfil those requests, and administrators can monitor and manage all activities. The platform improves coordination, reduces manual effort, and ensures timely support delivery.

## 1.2 Scope:

CareConnect is a scalable, microservice-based application with a React frontend and backend services developed using Spring Boot and Node.js. Each social service module operates independently and supports role-based access, notifications, and administrative analytics for efficient system management.

## 1.3 Definitions / Acronyms:

- **API:** Interface for communication between services
- **JWT:** Token-based user authentication
- **NGO:** Non-profit social organization
- **DB:** Data storage system
- **CI/CD:** Automated build and deployment process
- **Microservice:** Independent service for a specific function
- **API Gateway:** Single entry point for backend services
- **REST:** Standard for web service communication
- **Docker:** Tool to containerize applications
- **Kubernetes (K8s):** Manages and scales containers
- **RBAC:** Access control based on user roles

# 2. Overall Description

## 2.1 Product Perspective:

CareConnect is a microservices-based system where each service is independently deployable and maintains its own database, following polyglot persistence. An API Gateway acts as a single-entry point to route requests to backend services.

## 2.2 User Classes:

- **Admin:** Manages users, services, and system monitoring
- **NGO Volunteer:** Fulfils social support requests
- **Donor:** Provides donations and support
- **Beneficiary / Requester:** Raises help requests

## 2.3 Operating Environment:

Backend services are cloud-hosted on AWS EC2 or Render. The React frontend is deployed on Vercel. Databases used are MySQL (Auth, User, Care) and MongoDB (Blood, Orphanage).

## 2.4 Constraints & Assumptions:

The system requires internet access. Each microservice has its own database. JWT-based authentication and role-based access control are enforced across the platform.

# 3. System Features and Requirements

## 3.1 Functional Requirements

- User registration and login using JWT authentication
- Role-based access (Admin, NGO, Donor, Volunteer, Beneficiary)
- Submit help requests for Blood, Orphanage, Handicap, and Old-Age support
- View, accept, fulfil, and track request status
- Admin management of users, requests, and services
- Notification delivery via Email/SMS on request updates
- API Gateway routing and authentication validation

## 3.2 Non-Functional Requirements

- System shall support concurrent users with response time < 2 seconds
- Secure communication using HTTPS and encrypted passwords (bcrypt)
- Scalable microservices with independent deployment
- High availability (24/7) with fault tolerance
- Reliable data storage with regular backups

## 3.3 External Interface Requirements

- **User Interface:** Responsive web UI using **React + Tailwind**, role-based dashboards for Admin, NGO, Donor, Volunteer, and Beneficiary; works on desktop, tablet, and mobile.
- **Software Interface: REST APIs** via Spring Boot and Node.js; Swagger documentation for integration.
- **Database Interface:** Each microservice uses its own DB (**MySQL / MongoDB**) for isolation and scalability.
- **API Interface: JWT-secured REST endpoints** via centralized API Gateway for routing, authentication, and access control.

## 3.4 System Features

- **Microservices Architecture:** Independent services with **polyglot persistence**.
- **API Gateway:** Centralized routing, authentication, and security.
- **Containerization:** Dockerized services with **Kubernetes orchestration**.
- **CI/CD Pipeline:** GitHub Actions for automated build, test, and deployment.
- **Notification Service:** Modular service for **email, SMS, and push notifications**.

# 4. Other Requirements

## 4.1 Database Requirements

- Each microservice shall maintain its **own database** (MySQL or MongoDB) to ensure service independence.
- Databases shall support **ACID transactions** for critical operations and **replication/backups** for reliability.
- Data integrity, indexing, and optimized queries shall be implemented for efficient request handling.

## 4.2 Legal and Regulatory Requirements

- The system shall comply with **data protection laws** (e.g., GDPR/India Data Privacy Guidelines).
- Users' personal and medical information shall be **securely stored and encrypted**.
- The platform shall include **terms of service and privacy policies** for transparency and legal compliance.

## 4.3 Internationalization and Localization

- The system shall support **multi-language capability** for future scalability (e.g., English, Hindi).
- Date, time, currency, and number formats shall adapt to **user locale settings**.
- UI text and notifications shall be **externalized** to allow easy translation.

## 4.4 Risk Management

- **Security Risks:** Use HTTPS, JWT authentication, password hashing, and role-based access control.
- **Operational Risks:** Implement **monitoring, health checks, and automated alerts** for service downtime.
- **Data Risks:** Regular **database backups, replication, and disaster recovery plan**.
- **Scalability Risks:** Microservice architecture and container orchestration (Docker + Kubernetes) to handle high loads.

# 5. Appendices

## 5.1 Glossary

**API Gateway**: Single entry point routing requests to microservices with authentication and load balancing.

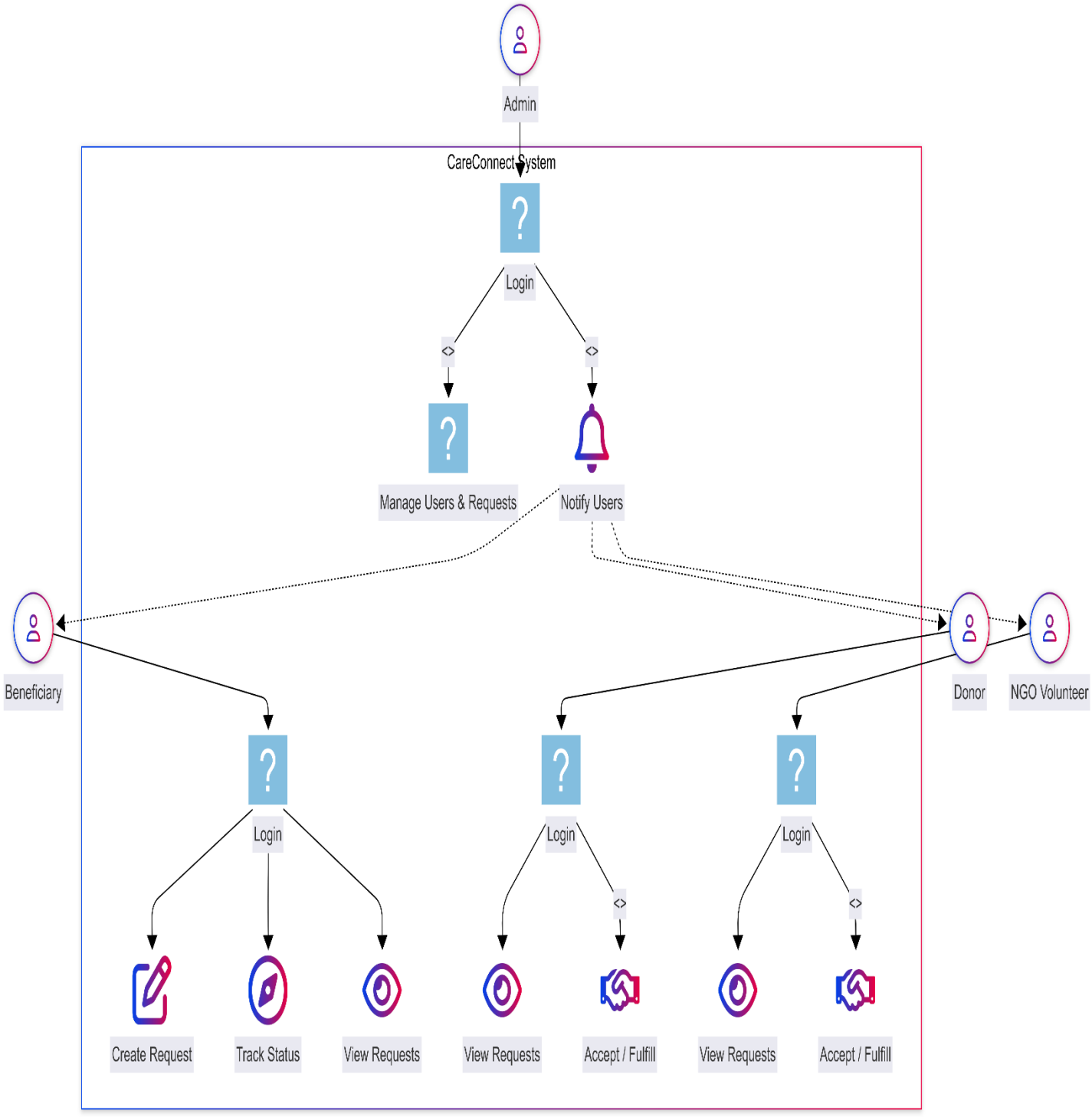
**JWT**: JSON Web Token for secure, stateless user authentication across services.

**Microservice**: Independent, deployable service handling specific functions like auth or notifications.

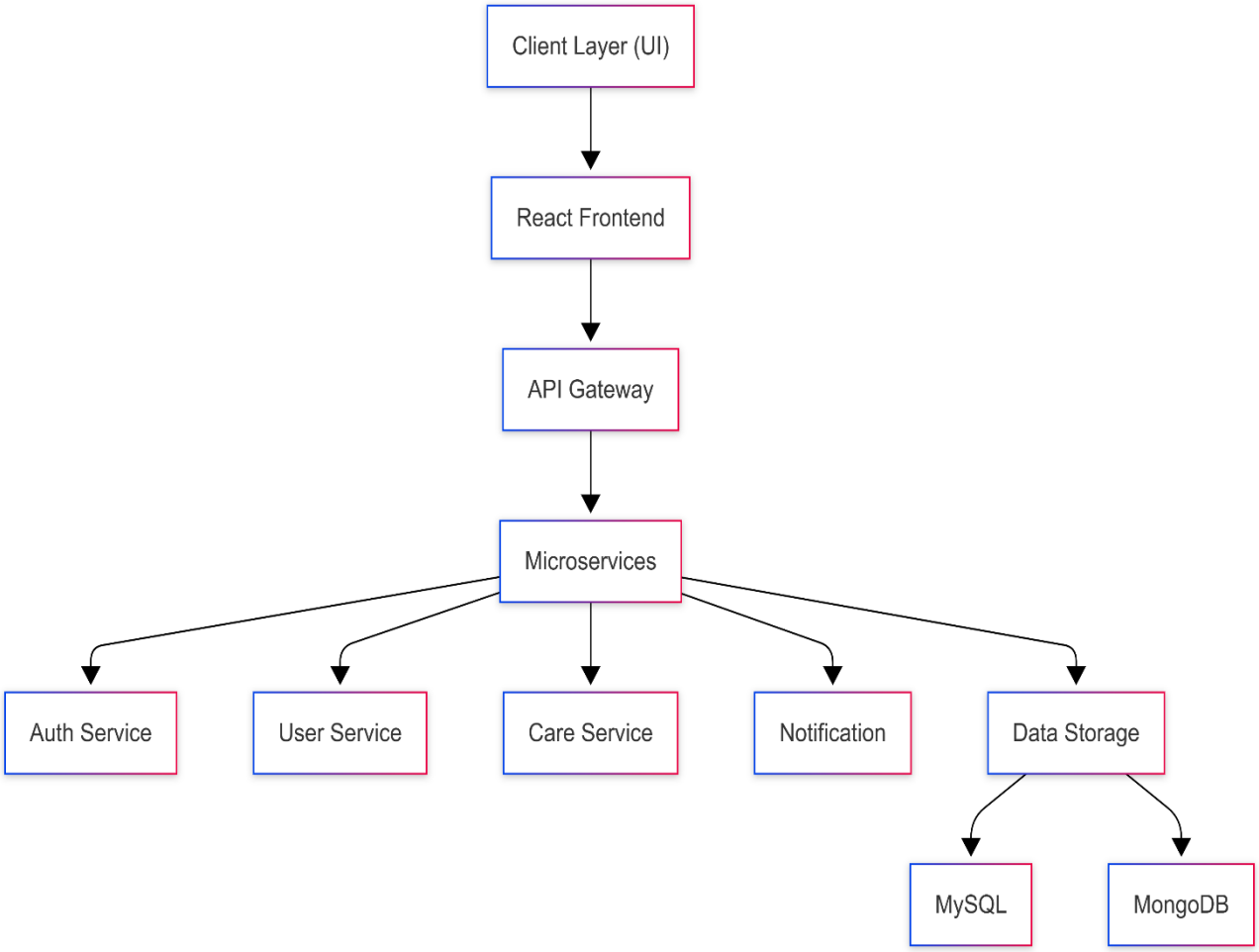
**Polyglot Persistence**: Using different databases (MySQL, MongoDB) per microservice for optimal data fit.

**RBAC**: Role-Based Access Control limiting features by user roles (Admin, Beneficiary, etc.).

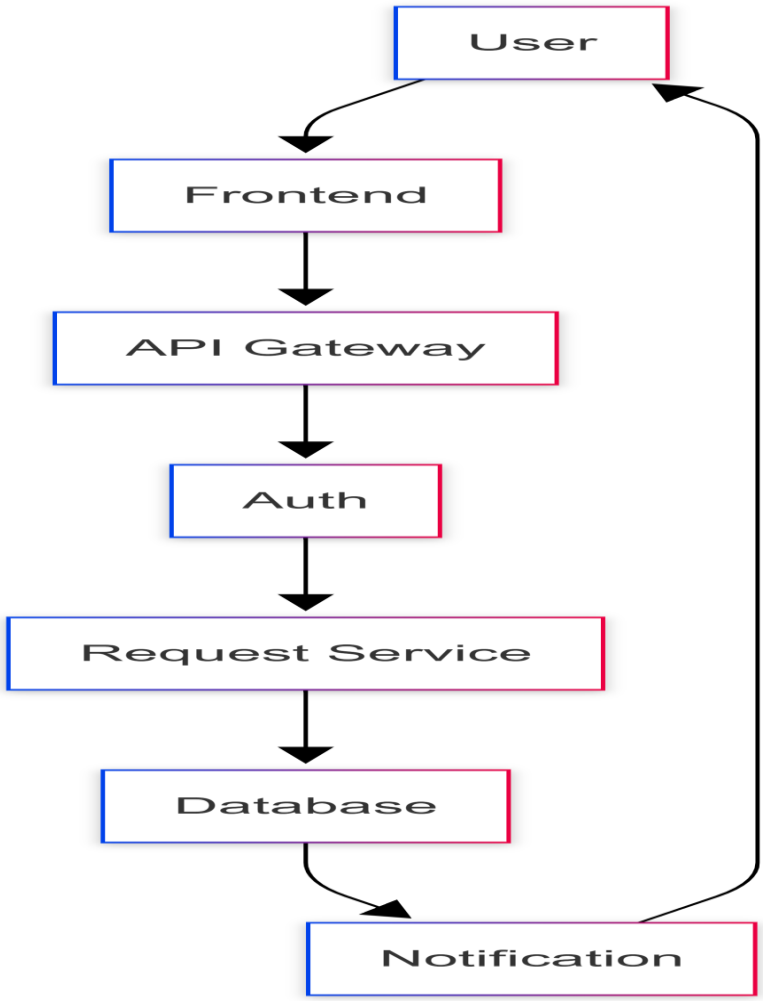
### 5.2.1 Use Case Diagram



### 5.2.2 High-Level Architecture Diagram



### 5.2.3 Sequence Diagram




## 5.3 TBD List

- Detailed database schemas for each microservice (MySQL/MongoDB tables/collections).
- Complete REST API endpoints with Swagger specs.
- CI/CD pipeline configuration (GitHub Actions YAML files).
- Kubernetes deployment manifests and Helm charts.
- Multi-language localization files (English/Hindi translations).
- Performance benchmarks for <2s response time under load.
- Disaster recovery procedures and backup schedules.
- Integration tests for API Gateway and notification flows.